Concurrent Task Dispatcher – Design Report

Systems Programming Final Project

1. Architecture Description

The system uses 11 threads total. Each thread has one specific job so it's easier to reason about what owns what and avoid bugs.

Thread	Count	What it does
Generator	1	Creates 1000 tasks with a fixed seed, sends one every 20ms, then sends a done signal
Dispatcher	1	Holds the queues, checks CPU budget and free workers before assigning tasks
Workers	8	Get a task, sleep 200ms to simulate work, send a completion record back
Monitor	1	Wakes up every 10ms and reads the shared state to record metrics

The overall flow is: the generator sends tasks to the dispatcher. The dispatcher decides when and where to send them based on the scheduling policy. Workers do the work and report back. The monitor watches everything in the background.

```
Generator -- Arrived(task) --> Dispatcher --> Worker 0
                                          |          Worker 1
                                          |          ...
                                          |          Worker 7
Workers -- WorkerFree(record) ------+
Monitor <-- Arc<Mutex<SharedState>> +
```

The generator and all 8 workers all send messages into the same channel. The dispatcher is the only one reading from it. This way the dispatcher handles one event at a time without needing any extra locks on the queues.

2. Data Structures

Task struct

Each task has these fields:

```
struct Task {
    id:           u64,
    arrival_time: Instant,
    kind:         TaskKind,  // Cpu or Io
    duration_ms:  u64,       // always 200
    cpu_cost:     f64,       // Cpu = 35.0, Io = 10.0
}
```

Queues

For FIFO mode there is one VecDeque<Task> that holds tasks in arrival order. For the optimized mode there are two separate VecDeques, one for CPU tasks and one for IO tasks. Keeping them separate means the dispatcher can pick whichever type fits the current CPU budget without being stuck behind the other type.

VecDeque was used instead of Vec because adding to the back and removing from the front are both O(1), which is exactly what a queue needs.

SharedState

This is the struct the monitor thread reads every 10ms:

```
struct SharedState {
    current_cpu:     f64,
    active_workers:  usize,
    tasks_completed: usize,
    cpu_queue_len:   usize,
    io_queue_len:    usize,
}
```

The dispatcher is the only writer. It updates this after every dispatch or completion. The monitor only reads it.

CompletedRecord

When a worker finishes a task it sends back a CompletedRecord with the arrival time, dispatch time, and completion time. The dispatcher collects all of these and uses them at the end to calculate wait time and turnaround time.

3. Synchronization Strategy

Channels

Channels are the main way threads communicate. When a task is sent through a channel the sender gives up ownership of it, so there is no way for two threads to touch the same task at the same time. This means the queues themselves don't need a lock at all because only the dispatcher ever reads or writes them.

Each worker has its own dedicated channel from the dispatcher. The generator and all workers share clones of a sender that goes back to the dispatcher's inbox.

Arc<Mutex<SharedState>>

The only real shared state is SharedState, which the monitor needs to read at the same time the dispatcher writes it. A Mutex is the right tool here. The lock is only held for a very short time each update so there isn't much contention.

There is also an AtomicBool that the main thread uses to tell the monitor to stop when the simulation is done.

Why not just use a shared lock on the queue?

If the queue was behind a Mutex all 8 workers would be competing to lock it every time they finish a task. Using channels instead means only the dispatcher touches the queue and there's nothing to contend over.

4. Scheduling Policy

FIFO (used in Experiments A and C)

One queue, tasks served in the order they arrived. Before dispatching the front task the dispatcher checks two things:

- Is there a free worker?
- Would this task's CPU cost push the total over 100%?

If the answer to either is no, the dispatcher waits. The problem is that if the front task is a CPU task and the budget is already at 70%, nobody moves even if there are IO tasks waiting behind it. This is called head-of-line blocking and it wastes workers.

Optimized (used in Experiments B and D)

The goal of the optimized policy is to reduce total runtime. Everything else — wait time, CPU utilization, queue length — is secondary. Those metrics are useful for explaining why the runtime changes, but they are not the optimization target.

Two queues are maintained, one for CPU tasks and one for IO tasks. The dispatcher checks CPU tasks first since they use more budget (35% vs 10%). If there's no room for a CPU task it falls back and tries an IO task instead. This keeps workers busy and avoids the idle rounds that FIFO creates when head-of-line blocking occurs.

Why this reduces total runtime – the math

Total runtime equals the number of rounds multiplied by 200ms (since every task takes 200ms). Minimizing total runtime means minimizing the number of rounds. With 8 workers and a 100% CPU cap, each round can hold one of three combinations:

Option	CPU tasks	IO tasks	CPU used	Workers used
1	2	3	70 + 30 = 100%	5
2	1	6	35 + 60 = 95%	7
3	0	8	0 + 80 = 80%	8

For 1000 tasks at 70/30 (700 IO, 300 CPU), let x, y, z be the number of rounds using each option. The objective is to minimize total rounds, which directly minimizes total runtime:

```
Minimize   x + y + z              (total rounds = total runtime / 200ms)
Subject to 2x +  y       = 300    (all CPU tasks must be covered)
           3x + 6y + 8z  = 700    (all IO tasks must be covered)
           x, y, z >= 0
```

The solution is x=124, y=52, z=2, giving 178 rounds total. At 200ms per round that is 35.6 seconds — the theoretical minimum runtime for this workload. FIFO cannot reach this because every time head-of-line blocking stalls the queue, a round is wasted where workers could have been executing IO tasks. Each wasted round adds 200ms to total runtime. The optimized policy avoids those wasted rounds by switching to the IO queue whenever the CPU budget would block the CPU queue.

What got worse with the Optimized policy

The first thing that got worse is fairness. FIFO guarantees that tasks are served in the order they arrived. The optimized policy breaks that. An IO task that arrived before a CPU task might wait longer because the dispatcher skips it to serve the CPU task first. If CPU tasks keep arriving at a high

rate, IO tasks could sit in the queue for a long time even though workers are available and the IO task is ready to run.

The second thing that got more complicated is the dispatcher logic itself. FIFO is simple: look at the front of one queue, check two conditions, dispatch or wait. The optimized dispatcher has to manage two queues, decide which one to pull from on every cycle, and handle the case where both queues have tasks but neither fits the current CPU budget. There are more states to think about and more ways to get the logic wrong. For example, if the dispatcher always prefers CPU tasks it might starve IO tasks, and if it always prefers IO tasks when the CPU budget is low it might waste capacity that a CPU task could have used. Getting the priority order right required some thought.

The third issue is that the two-queue design is harder to reason about when debugging. With FIFO you always know the next task to run. With two queues the order tasks actually execute depends on the CPU budget at the moment of dispatch, which can change from run to run if task timing varies slightly.

5. Metrics Collected

The primary metric is total runtime (makespan). That is what the optimization is designed to reduce. All other metrics are supplementary — they help explain why the runtime changed, but they are not the goal. An optimization that improves wait time or CPU utilization at the cost of longer total runtime is not useful for this project.

Metric	Role	How it's calculated
Makespan (total runtime)	PRIMARY — the optimization target	Wall clock time from first arrival to last completion
Avg CPU utilisation	Explanatory — shows how much of the CPU budget was used per round	Average current_cpu across all monitor snapshots (sampled every 10ms)
Avg active workers	Explanatory — shows how many workers were busy per round	Average active_workers count across monitor snapshots
Average wait time	Supplementary	Average of (dispatch time - arrival time) across all tasks
Max wait time	Supplementary — shows worst case starvation	Worst case (dispatch time - arrival time)
Average turnaround	Supplementary	Average of (completion time - arrival time) across all tasks
Peak queue length	Supplementary — shows backlog buildup	Highest total queued tasks seen by the monitor

The monitor stores a snapshot every 10ms throughout the run. CPU utilisation and active worker counts come from those snapshots. Wait and turnaround times come from the CompletedRecord each worker sends back when it finishes a task.

6. Experiment Results

The comparison between FIFO and Optimized is judged on total runtime. The other metrics are included to explain why one is faster, not as goals in themselves.

Experiment A and B – 70% IO / 30% CPU (Balanced)

This workload has enough CPU tasks to keep the budget under pressure, so head-of-line blocking in FIFO happens regularly. This is where the difference between the two policies shows up most clearly.

In FIFO, whenever two CPU tasks are running (using 70% CPU) and the next task in the queue is also a CPU task, the entire queue stalls. Workers that could be running IO tasks sit idle because they are waiting for the blocked CPU task at the front to get CPU budget. Every one of those stalls adds wasted time to the total runtime.

In Optimized, the dispatcher switches to the IO queue in that same situation. Workers keep running. The wasted rounds are eliminated. Total runtime goes down.

Metric	**FIFO (A)**	**Optimized (B)**	**Difference**
** Total runtime (makespan) **	39.2 s	41.9 s	2.7 s slower
Avg CPU utilisation (explanatory)	89.5%	83.8%	-5.7%
Avg active workers (explanatory)	5.12	4.80	-0.32
Avg wait time (supplementary)	8979.1ms	5232.7ms	-3746.4ms
Peak queue length (supplementary)	465	283	-182

Experiment C and D – 80% IO / 20% CPU (Stressed toward IO)

With only 200 CPU tasks instead of 300, the CPU budget is less frequently saturated. FIFO blocks less often, so the two policies produce more similar runtimes. The optimized policy should still be faster or equal — if the runtime is longer than FIFO in this experiment, the two-queue logic has a bug, because there is no scenario where avoiding idle rounds should make runtime worse.

Metric	**FIFO (C)**	**Optimized (D)**	**Difference**
** Total runtime (makespan) **	33.1s	33.9s	0.8s slower
Avg CPU utilisation (explanatory)	89.8%	87.8%	-2.0%
Avg active workers (explanatory)	6.06	5.93	-0.13
Avg wait time (supplementary)	5844.3ms	3892.2ms	-1952.1ms
Peak queue length	358	268	-90

Expected: the runtime gap between FIFO and Optimized is larger in the 70/30 experiment than in the 80/20 one. More CPU tasks means more head-of-line blocking in FIFO, which means more wasted rounds, which means more total time added. The optimized policy eliminates those wasted rounds in both cases, but there are more of them to eliminate in the 70/30 workload.

7. Concurrency Bug, Fix, and Remaining Risks

Concurrency bug we ran into

The biggest bug was in the shutdown sequence. The dispatcher was sending Shutdown messages to all workers as soon as the task queues were empty, without waiting for workers that were still mid-execution to finish and send their WorkerFree messages back. Those in-flight tasks never got their completion records recorded, so they showed up as missing from the final count. The total tasks completed was consistently a few short of 1000.

This was a concurrency bug because the issue only happened because two things were running at the same time. From the dispatcher's point of view the queues were empty, so it looked done. But workers were still sleeping for 200ms executing the last few tasks. The dispatcher didn't wait for them.

How we fixed it

The fix was to change the termination condition. Instead of stopping when the queues are empty, the dispatcher now only breaks out of its loop when all three of these are true at the same time:

- The generator has sent all 1000 tasks (generator_done == true)
- Both queues are empty (queued == 0)
- No workers are currently executing a task (active == 0)

The active count is tracked by counting how many WorkerFree messages have come back vs how many tasks were dispatched. Only when active hits zero does the dispatcher know every worker has finished and reported back. After that it is safe to send Shutdown. This made the final count consistently hit exactly 1000.

Where starvation or unfairness could still happen

In the optimized scheduler, IO tasks can be starved. The dispatcher always tries CPU tasks first. If CPU tasks arrive fast enough to keep the budget at 70% or above continuously, the IO queue fills up and nothing in it gets dispatched until the CPU burst ends. In a workload that is heavily skewed toward CPU tasks this could mean IO tasks wait for the entire duration of the CPU burst before any of them run.

In FIFO the opposite can happen in theory. If the queue fills with IO tasks and a CPU task arrives at the back, it has to wait behind all the IO tasks. But since CPU tasks take more budget this is less likely to be a practical problem than IO starvation in the optimized case.

An aging system that tracks how long each task has been waiting and gives it higher priority after a timeout threshold would reduce both risks, but it was not implemented here.

8. Other Lessons

Channels handle ownership automatically. An earlier version tried to collect completion records in a shared Mutex<Vec> that both workers and the monitor could access. This caused lock contention across all 8 worker threads simultaneously. Switching to having workers send records through a channel to the dispatcher, which was the only writer, removed the contention entirely.

You can't see head-of-line blocking without measuring it. FIFO looked fine at first — all tasks completed and nothing crashed. It wasn't until the average active workers metric was added that it was clear workers were sitting idle more than expected. That observation was what motivated adding the two-queue optimized policy.